

Inbound-Outbound Management System

System Design Document

Contents

1	Understanding the Problem	3
2	Functional Requirements	3
2.1	Core Requirements	3
2.2	Out of Scope	3
3	Non-Functional Requirements	3
3.1	Core Requirements	3
4	The Set Up	4
5	Core Entities	4
6	API or System Interface	5
6.1	Authentication APIs	5
6.2	Verification APIs	5
6.3	Logs and Dashboard APIs	6
6.4	Admin APIs	6
6.5	Offline Sync APIs	6
7	High-Level Design (numbered)	7
7.1	1. Staff authentication and access-protected workflows	7
7.2	2. Security staff scan barcodes for entry and exit	8
7.3	3. The system creates movement logs and active state	8
7.4	4. Manual verification handles missing or damaged barcodes	8
7.5	5. Dashboard and logs support monitoring	8
7.6	6. Alerts surface risky movement attempts	8
8	Low-Level Design	9
8.1	Core Enums	9

8.2	Request and Response Objects	9
8.3	Domain Classes and Responsibilities	10
8.4	Service Classes	10
8.5	Repository Interfaces	11
8.6	Presence State Transitions	11
9	Data Model	11
9.1	roles	11
9.2	users	12
9.3	shiftPolicies	12
9.4	employees	12
9.5	visitors	12
9.6	hardwareItems	13
9.7	checkpoints	13
9.8	scannerDevices	13
9.9	barcodeCredentials	13
9.10	offlineSyncBatches	14
9.11	movementTransactions	14
9.12	activePresence	15
9.13	alerts	15
9.14	Entity-Relationship Diagram	16
9.15	Broad version	16
9.16	Simple version	17
9.17	Architecture Design	18
9.17.1	Security staff use case	18
9.17.2	Admin dashboard use case	18
10	Potential Deep Dives	19
10.1	1. How are duplicate scans and conflicting active entries prevented?	19
10.2	2. How do we support offline verification without corrupting audit logs?	19
10.3	3. How do we keep authentication and local storage secure?	19
10.4	4. What happens when scanners, power, or backend services fail?	19
10.5	5. Why are users and employees separate but linked?	19

1 Understanding the Problem

The Inbound-Outbound Management System manages the entry and exit of employees, visitors, and hardware items digitally. It replaces paper-based verification, manual registers, and informal admin monitoring with a traceable workflow.

The core idea is that every inward or outward movement should become a verified, timestamped, auditable transaction. Security staff use barcode scanners or controlled manual forms at checkpoints. Administrators monitor movement logs, alerts, and operational summaries from a dashboard.

2 Functional Requirements

2.1 Core Requirements

1. Security staff should be able to verify employee and hardware movement by scanning a barcode, and verify visitor movement through an approved manual or visitor-record lookup fallback.
2. The system should create entry and exit movement records with server-side timestamps and real-time status updates.
3. The system should detect invalid, duplicate, expired, restricted, incomplete, or mismatched scans and show immediate feedback.
4. Administrators should be able to monitor logs, alerts, and activity summaries from a dashboard.
5. Security staff should be able to continue limited verification when the network or backend is unavailable, then sync safely when connectivity returns.

2.2 Out of Scope

1. Physical door, turnstile, or gate control. The system records, verifies, and alerts, but does not physically block movement.
2. Facial recognition or biometric verification.
3. Advanced analytics beyond basic filtering, summaries, and operational counters.
4. Investigation of illegal activity beyond rule-based alerts such as restricted exit, duplicate scan, off-hours attempt, or repeated failures.

3 Non-Functional Requirements

3.1 Core Requirements

1. Barcode verification should complete within 3 seconds for normal online operations.
2. The same employee, visitor, or hardware item should not have conflicting active entry or exit state.
3. Every accepted, rejected, manual, offline, sync, and admin action must be traceable.

4. Authentication, authorization, and role-based access control are required.
5. Audit data should remain trustworthy even when staff, employees, visitors, devices, or hardware items are deactivated later.
6. Offline verification must be bounded, clearly labelled, replayed through server-side validation, and surfaced for review when conflicts occur.
7. The system should target high availability during operating hours, with planned maintenance communicated in advance and degraded offline verification available when backend services are unavailable.
8. The database and uploaded sync batches must be backed up regularly, with documented recovery procedures and periodic restore testing.
9. Movement logs, audit trails, alerts, and visitor records must have a defined retention period that satisfies operational, legal, and company policy requirements.
10. Personally identifiable information must be protected through least-privilege access, encryption in transit, encryption at rest where supported, and masking or minimization in logs and exports.
11. The system must provide observability through structured logs, health checks, metrics, and alerts for failed scans, scanner disconnects, offline sync failures, latency, and error rates.
12. The system should define expected scan volume, peak checkpoint throughput, concurrent operator count, and offline batch size so performance and database capacity can be tested against realistic load.

4 The Set Up

5 Core Entities

1. User: A login identity for a system operator, such as an administrator or security staff member.
2. Role: Permission set assigned to a user, such as admin or security staff.
3. Employee: A person whose movement can be tracked. If the employee can also log in, the employee row links to a user row through nullable `user_id`.
4. Visitor: A temporary person whose movement can be tracked and who may be hosted by an employee.
5. Hardware Item: A physical component or asset that can enter or leave.
6. Barcode Credential: A unique barcode mapped to exactly one employee or hardware item. Visitors do not use barcode credentials. This is modelled with nullable foreign keys and a check constraint, not an unenforced polymorphic `subject_id`.
7. Checkpoint: Physical station such as a gate or security desk.
8. Scanner Device: Registered scanning device assigned to a checkpoint.
9. Movement Transaction: Immutable record of a verification attempt and resulting entry or exit decision.
10. Active Presence: Current state for an employee, visitor, or hardware item.

11. Verification Result: approved, pending, denied, duplicate, expired, restricted, or manual_review.
12. Alert: Security warning produced by invalid scans, duplicate scans, restricted movement, repeated failures, or sync conflicts.
13. Offline Sync Batch: Batch of offline scans uploaded after connectivity returns.
14. Shift Policy: Optional rules that define allowed employee movement windows.

6 API or System Interface

6.1 Authentication APIs

```
POST /auth/login
POST /auth/logout
GET /auth/session
GET /auth/oidc/callback
```

Authentication should use OIDC for SSO. The API derives the authenticated `user_id`, role, permissions, and session server-side.

6.2 Verification APIs

```
POST /verification/entry
```

Body:

```
{
  "employeeBarcode": "optional",
  "visitorId": "optional uuid",
  "hardwareBarcode": "optional",
  "checkpointId": "uuid",
  "scanSource": "scanner|manual|offline",
  "idempotencyKey": "client-generated-uuid"
}
```

```
POST /verification/exit
```

Body:

```
{
  "employeeBarcode": "optional",
  "visitorId": "optional uuid",
  "hardwareBarcode": "optional",
  "checkpointId": "uuid",
  "scanSource": "scanner|manual|offline",
  "idempotencyKey": "client-generated-uuid"
}
```

```
POST /verification/manual
```

Body:

```
{
  "employeeId": "optional uuid",
  "visitorId": "optional uuid",
  "hardwareItemId": "optional uuid",
  "employeePin": "optional",
  "movementType": "entry|exit",
  "checkpointId": "uuid",
  "reason": "string",
  "idempotencyKey": "client-generated-uuid"
}
```

The client must not send trusted timestamps, verification status, or user role. Those are derived server-side. At least one of employee, visitor, or hardware identity must be supplied. Visitor movement uses a `visitorId` from an approved lookup or manual verification flow, not a barcode credential. An employee or visitor may also be associated with a hardware item when a person is carrying an asset.

6.3 Logs and Dashboard APIs

```
GET /logs?employeeId=&visitorId=&hardwareItemId=&barcode=&status=&checkpointId=&from=
=&to=&page=
GET /logs/:id
GET /dashboard/summary?from=&to=&checkpointId=
GET /dashboard/recent-activity?limit=&checkpointId=
```

6.4 Admin APIs

```
GET /employees
POST /employees
GET /visitors
POST /visitors
GET /hardware-items
POST /hardware-items
GET /barcode-credentials
POST /barcode-credentials
GET /alerts
PATCH /alerts/:id
```

6.5 Offline Sync APIs

```
POST /sync/offline-batch
```

Body:

```
{
  "deviceId": "uuid",
  "checkpointId": "uuid",
  "batchId": "uuid",
  "capturedEvents": [],
}
```

```
"clientCacheVersion": "string"
}
```

```
GET /sync/bootstrap?checkpointId=&deviceId=
```

The bootstrap endpoint returns a bounded, signed cache of employees, visitors, hardware items, restrictions, shift rules, and checkpoint rules for limited offline operation.

7 High-Level Design (numbered)

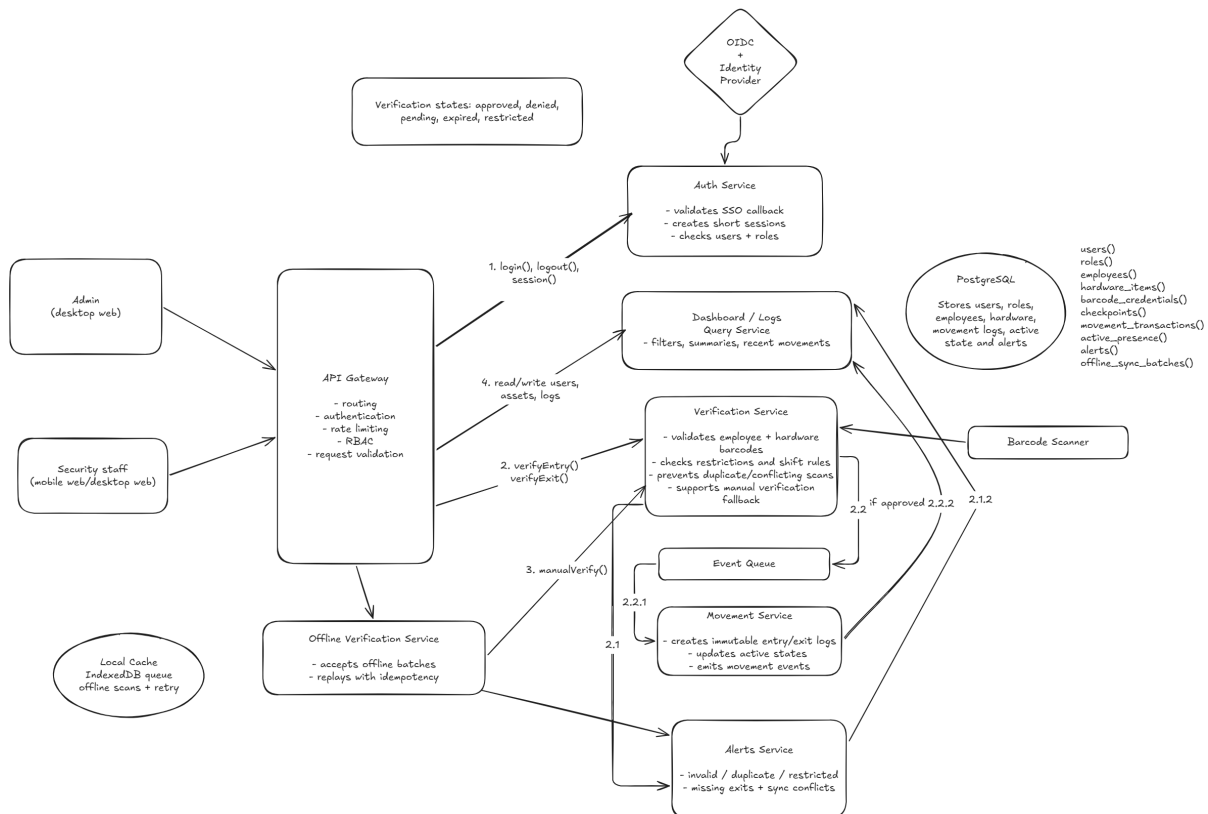


Figure 1: High-Level System Design

7.1 1. Staff authentication and access-protected workflows

The operator opens the web application and authenticates through OIDC. The API Gateway validates the session and assigns the role through RBAC. Admin users get dashboard and management permissions, while security staff get checkpoint verification permissions.

Core components:

1. Security Desk Client
2. API Gateway
3. OIDC Service connected to an identity provider
4. Role-based access control

5. Session tracking

7.2 2. Security staff scan barcodes for entry and exit

The scanner module reads employee or hardware barcodes. Visitor movement is verified through visitor lookup or the manual verification flow. The client sends the scan payload to the verification service with an idempotency key. The service validates barcode format, resolves the credential through enforced foreign keys, checks restrictions and active state, then asks the movement service to process the transaction.

The result is returned as approved, pending, denied, duplicate, expired, restricted, or manual_review. The UI renders a status badge, verification card, and toast notification.

7.3 3. The system creates movement logs and active state

The movement service writes an immutable movement transaction and updates active presence in the same database transaction. Database constraints prevent conflicting active state for the same employee, visitor, or hardware item.

7.4 4. Manual verification handles missing or damaged barcodes

When a barcode is damaged, forgotten, or unreadable, the staff uses the manual method. The verification service validates the supplied employee, visitor, or hardware identity, reason, checkpoint permissions, and active movement conflicts.

Manual approvals are labelled separately so that administrators can review their authenticity later.

7.5 5. Dashboard and logs support monitoring

The dashboard reads from a query-optimized path. For small scale, indexed tables and summary queries are enough. As logs grow, events from the movement service can feed OpenSearch or Elasticsearch for filtering and a dashboard read model for counters such as total entries, exits, alerts, active presence, and success rate.

7.6 6. Alerts surface risky movement attempts

The alerts service consumes movement and verification events. It creates alerts for:

1. Unregistered IDs
2. Expired visitor pass
3. Restricted hardware exit
4. Duplicate scans
5. Repeated failed attempts
6. Missing exit
7. Outside-working-hours attempts

8. Scanner disconnects
9. Offline sync conflicts

8 Low-Level Design

This low-level design follows the usual LLD/OOD structure: enums, domain models, service responsibilities, class relationships, state transitions, concurrency handling, and design patterns.

8.1 Core Enums

1. `MovementType`: `entry`, `exit`.
2. `ScanSource`: `scanner`, `manual`, `offline`.
3. `SubjectType`: `employee`, `visitor`, `hardware_item` for movement subjects. Barcode credentials only support `employee` and `hardware_item`.
4. `VerificationStatus`: `approved`, `pending`, `denied`, `duplicate`, `expired`, `restricted`, `manual_review`.
5. `PresenceState`: `inside`, `outside`.
6. `AlertSeverity`: `low`, `medium`, `high`, `critical`.
7. `AlertStatus`: `open`, `acknowledged`, `resolved`.
8. `OfflineBatchStatus`: `received`, `processing`, `processed`, `conflict`, `failed`.

8.2 Request and Response Objects

1. `VerificationRequest`: API-level request for entry or exit verification. It carries `movementType`, `employeeBarcode`, `visitorId`, `hardwareBarcode`, `checkpointId`, `scanSource`, and `idempotencyKey`.
2. `ManualVerificationRequest`: API-level request for manual approval. It carries `employeeId`, `visitorId`, `hardwareItemId`, `employeePin`, `movementType`, `checkpointId`, `reason`, and `idempotencyKey`.
3. `SubjectRef`: normalized subject reference after request parsing. It contains optional `employeeId`, `visitorId`, and `hardwareItemId`, with validation rules that allow one person plus one hardware item.
4. `VerificationContext`: internal object passed through validation. It includes the operator, checkpoint, source, resolved subject, active presence, credential status, restriction flags, and shift policy.
5. `VerificationDecision`: output from verification. It includes `status`, `failureReason`, `subjectRef`, `checkpointId`, `movementType`, `source`, `idempotencyKey`, and any alert hints.
6. `CapturedOfflineEvent`: offline event from a scanner device. It includes local event id, movement type, barcode hash or visitor id, checkpoint id, local captured time, local decision, cache version, and idempotency key.
7. `SyncResult`: result of replaying offline events. It includes accepted count, rejected count, conflict count, created transaction ids, and alert ids.

8.3 Domain Classes and Responsibilities

1. **User**: represents login identity and role.
2. **Role**: owns permission names and role checks.
3. **Employee**: represents a tracked employee and optional login linkage.
4. **Visitor**: represents a scheduled or active visitor. Visitors do not have barcode credentials.
5. **HardwareItem**: represents a tracked asset and its restriction state.
6. **BarcodeCredential**: maps one barcode hash to one employee or one hardware item.
7. **Checkpoint**: represents an entry, exit, or both-direction physical station.
8. **ScannerDevice**: represents a registered scanner assigned to a checkpoint.
9. **MovementTransaction**: immutable audit record of a verification attempt.
10. **ActivePresence**: current inside/outside state for one employee, visitor, or hardware item.
11. **Alert**: warning created by risky, invalid, duplicate, or conflicting movement.
12. **OfflineSyncBatch**: batch container for offline events uploaded by a device.
13. **ShiftPolicy**: rule set for allowed employee movement windows.

8.4 Service Classes

1. **AuthService**: validates sessions, resolves the current operator, and enforces RBAC.
2. **VerificationService**: orchestrates request normalization, credential resolution, policy evaluation, movement recording, and response mapping.
3. **BarcodeCredentialService**: hashes incoming barcode values, finds active credentials, and rejects expired or revoked credentials.
4. **VisitorLookupService**: validates visitor ids, host relationship, status, expiry, and allowed movement.
5. **PolicyEvaluator**: runs verification rules in a consistent order and returns the final decision.
6. **MovementService**: writes immutable movement transactions and updates active presence inside one database transaction.
7. **ActivePresenceService**: reads and transitions current presence state for a subject.
8. **AlertService**: creates alerts for failed scans, restricted attempts, repeated failures, scanner disconnects, and offline conflicts.
9. **OfflineSyncService**: stores offline batches, replays captured events through verification rules, detects conflicts, and produces sync results.
10. **DashboardQueryService**: provides summary counters and recent activity from indexed movement and alert data.
11. **AuditLogService**: records security-sensitive actions such as admin changes, manual verification, sync resolution, and alert updates.

8.5 Repository Interfaces

1. **UserRepository**: find users and roles by session or id.
2. **EmployeeRepository**: find employees by id, code, or linked user.
3. **VisitorRepository**: find visitors by id and update visitor status when movement completes.
4. **HardwareItemRepository**: find hardware items by id or asset code.
5. **BarcodeCredentialRepository**: find active credentials by barcode hash.
6. **CheckpointRepository**: find active checkpoints and registered scanner devices.
7. **MovementTransactionRepository**: insert immutable movement transactions and query logs.
8. **ActivePresenceRepository**: lock, read, and update subject presence rows.
9. **AlertRepository**: create, acknowledge, and resolve alerts.
10. **OfflineSyncBatchRepository**: insert sync batches and update processing status.

8.6 Presence State Transitions

Current State	Movement Type	Normal Result	Notes
outside	entry	approved	Creates entry transaction and changes state to inside.
inside	exit	approved	Creates exit transaction and changes state to outside.
inside	entry	duplicate	Existing active presence already shows the subject inside.
outside	exit	duplicate or denied	The subject is not currently inside. Use denied if this indicates a rule violation.
unknown	entry	approved or manual_review	Used when presence was never initialized. Hardware or visitor state may require review.
unknown	exit	manual_review	Exit without known entry should be reviewed.

9 Data Model

The data model is shown as design-level tables. Each table lists the important rows/fields, primary key, foreign-key relationships, and important rules without using database-specific syntax.

9.1 roles

Field	Key / Relationship	Notes
id	Primary key	Unique role identifier
name	Unique	Role name such as admin or security_staff

Field	Key / Relationship	Notes
permissions		Permission list for the role

9.2 users

Field	Key / Relationship	Notes
id	Primary key	Login identity identifier
roleId	Foreign key to roles.id	Defines system permissions
email	Unique	Used for login/SSO mapping
displayName		Human-readable name
status		active or disabled

9.3 shiftPolicies

Field	Key / Relationship	Notes
id	Primary key	Unique shift policy identifier
name	Unique	Policy name
rules		Shift timing and movement rules
status		active or disabled

9.4 employees

Field	Key / Relationship	Notes
id	Primary key	Unique employee identifier
userId	Foreign key to users.id, nullable, unique when present	Links an employee to a login user if they can access the system
shiftPolicyId	Foreign key to shiftPolicies.id, nullable	Employee movement timing policy
employeeCode	Unique	Company employee code
name		Employee name
department		Employee department
status		active, inactive, or suspended
createdAt		Created timestamp

9.5 visitors

Field	Key / Relationship	Notes
id	Primary key	Unique visitor identifier
hostEmployeeId	Foreign key to employees.id, nullable	Employee responsible for the visitor
name		Visitor name

Field	Key / Relationship	Notes
organization		Visitor company or organization
purpose		Reason for visit
status		scheduled, checked_in, checked_out, expired, or cancelled
createdAt		Created timestamp

9.6 hardwareItems

Field	Key / Relationship	Notes
id	Primary key	Unique hardware item identifier
assetCode	Unique	Internal asset code
serialNumber	Unique when available	Manufacturer or inventory serial number
type		Hardware category
ownerDepartment		Department that owns the item
status		active, retired, lost, or maintenance
restrictedExit		Whether exit requires restriction checks
createdAt		Created timestamp

9.7 checkpoints

Field	Key / Relationship	Notes
id	Primary key	Unique checkpoint identifier
name	Unique	Checkpoint name
location		Physical location (front gate, back gate)
type		entry, exit, or both (as our office has another door)
status		active or disabled

9.8 scannerDevices

Field	Key / Relationship	Notes
id	Primary key	Unique scanner device identifier
checkpointId	Foreign key to checkpoints.id	Checkpoint where the scanner is registered
deviceCode	Unique	Device inventory or registration code
status		active, disabled, or lost
lastSeenAt		Last time the device connected

9.9 barcodeCredentials

Field	Key / Relationship	Notes
id	Primary key	Unique credential identifier
barcodeValueHash	Unique	Hashed barcode value, not the raw barcode
subjectType		employee or hardware_item
employeeId	Foreign key to employees.id, nullable	Present only for employee credentials
hardwareItemId	Foreign key to hardwareItems.id, nullable	Present only for hardware credentials
status		active, expired, or revoked

9.10 offlineSyncBatches

Field	Key / Relationship	Notes
id	Primary key	Unique offline batch identifier
deviceId	Foreign key to scannerDevices.id	Device that uploaded the batch
checkpointId	Foreign key to checkpoints.id	Checkpoint where events were captured
status		received, processing, processed, conflict, or failed
receivedAt		Server receive time
resolvedAt		Time conflicts were resolved, if any
conflictCount		Number of sync conflicts found

9.11 movementTransactions

Field	Key / Relationship	Notes
id	Primary key	Unique movement transaction identifier
movementType		entry or exit
employeeId	Foreign key to employees.id, nullable	Employee involved in the movement
visitorId	Foreign key to visitors.id, nullable	Visitor involved in the movement
hardwareItemId	Foreign key to hardwareItems.id, nullable	Hardware item involved in the movement
checkpointId	Foreign key to checkpoints.id	Checkpoint where movement was recorded
operatorUserId	Foreign key to users.id	Security staff or admin who operated the system
verificationStatus		approved, pending, denied, duplicate, expired, restricted, or manual_review
failureReason		Reason when verification is not approved
source		scanner, manual, or offline
idempotencyKey	Unique	Prevents duplicate transactions from retries
occurredAt		Trusted server-side event time
offlineBatchId	Foreign key to offlineSyncBatches.id, nullable	Present for synced offline events

Rules:

1. At least one subject field must be present.
2. `employeeId` and `visitorId` should not both be present in the same movement.
3. A movement can include a person subject plus a hardware item when someone carries an asset.
4. Movement transactions should be immutable after creation.

9.12 activePresence

Field	Key / Relationship	Notes
<code>id</code>	Primary key	Unique active presence identifier
<code>employeeId</code>	Foreign key to <code>employees.id</code> , nullable	Present only for employee presence
<code>visitorId</code>	Foreign key to <code>visitors.id</code> , nullable	Present only for visitor presence
<code>hardwareItemId</code>	Foreign key to <code>hardwareItems.id</code> , nullable	Present only for hardware presence
<code>checkpointId</code>	Foreign key to <code>checkpoints.id</code>	Last checkpoint that updated the state
<code>movementTransactionId</code>	Foreign key to <code>movementTransactions.id</code>	Movement that produced this state
<code>state</code>		inside or outside
<code>enteredAt</code>		Time the subject entered, when applicable

Rules:

1. Exactly one of `employeeId`, `visitorId`, or `hardwareItemId` must be present.
2. Only one active presence row should exist per employee, visitor, or hardware item.
3. Active presence must be updated in the same transaction as the movement transaction.

9.13 alerts

Field	Key / Relationship	Notes
<code>id</code>	Primary key	Unique alert identifier
<code>severity</code>		low, medium, high, or critical
<code>alertType</code>		Type of security or sync warning
<code>movementTransactionId</code>	Foreign key to <code>movementTransactions.id</code> , nullable	Movement that caused the alert, if applicable
<code>status</code>		open, acknowledged, or resolved
<code>message</code>		Human-readable alert message
<code>createdAt</code>		Created timestamp
<code>resolvedBy</code>	Foreign key to <code>users.id</code> , nullable	User who resolved the alert
<code>resolvedAt</code>		Resolution timestamp

9.14 Entity-Relationship Diagram

9.15 Broad version

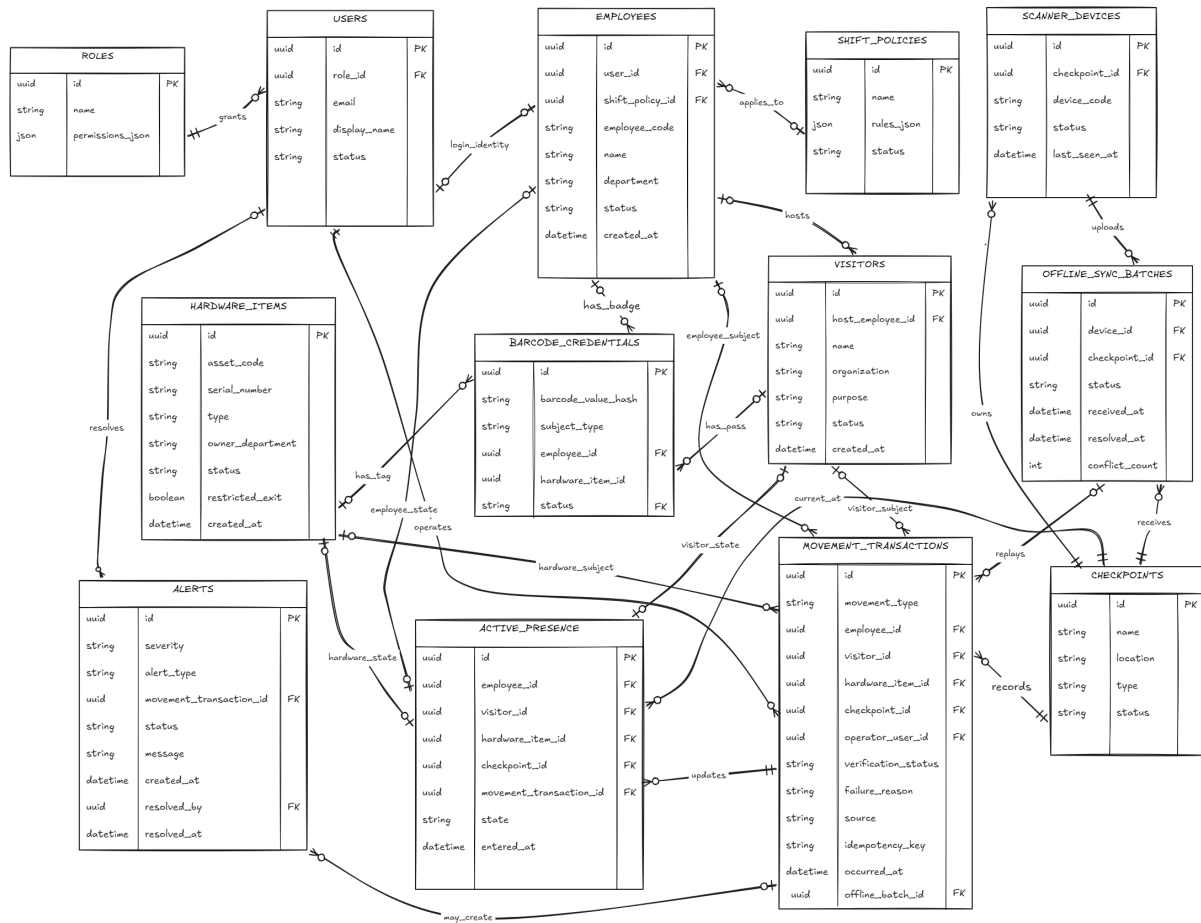


Figure 2: Entity-Relationship Diagram: Broad Version

9.16 Simple version

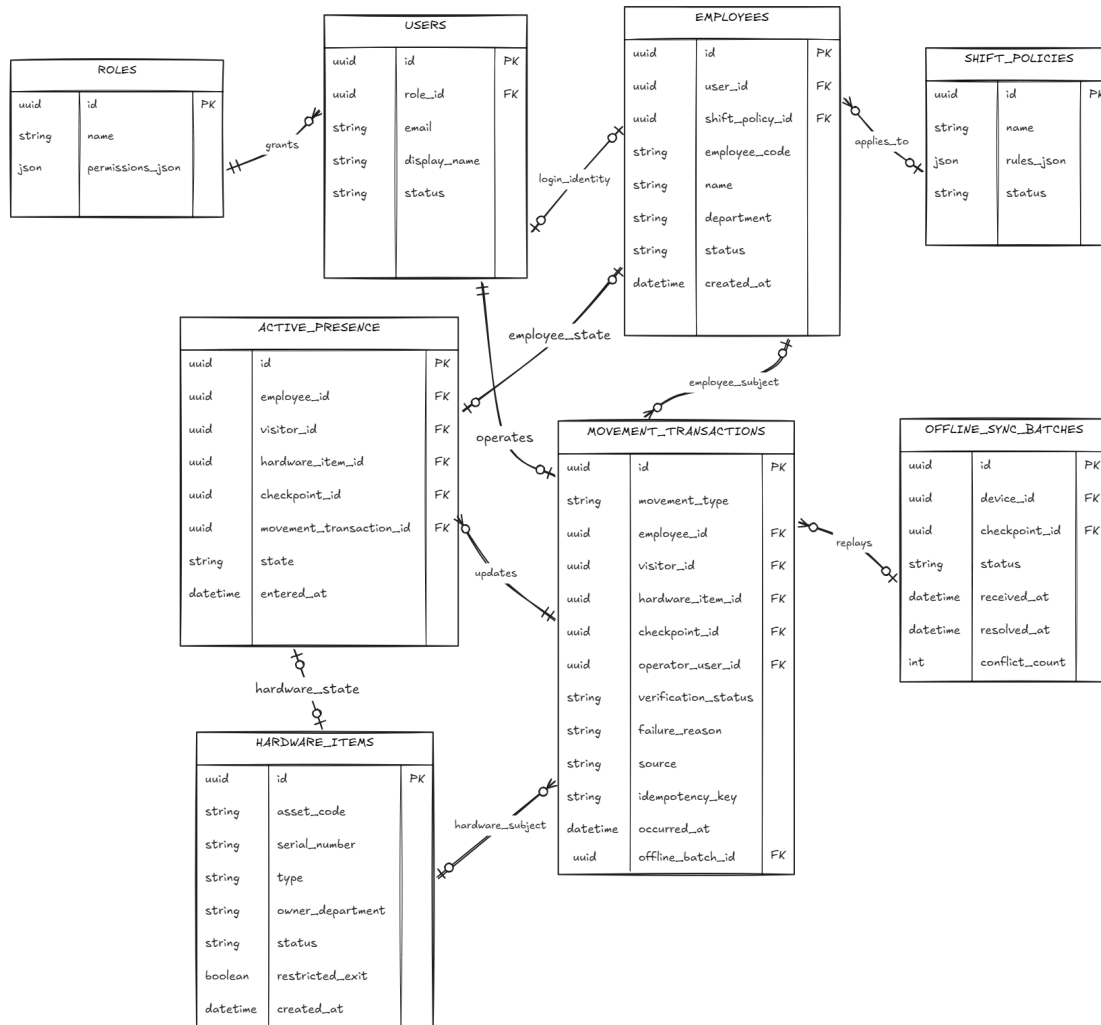


Figure 3: Entity-Relationship Diagram: Simple Version

9.17 Architecture Design

9.17.1 Security staff use case

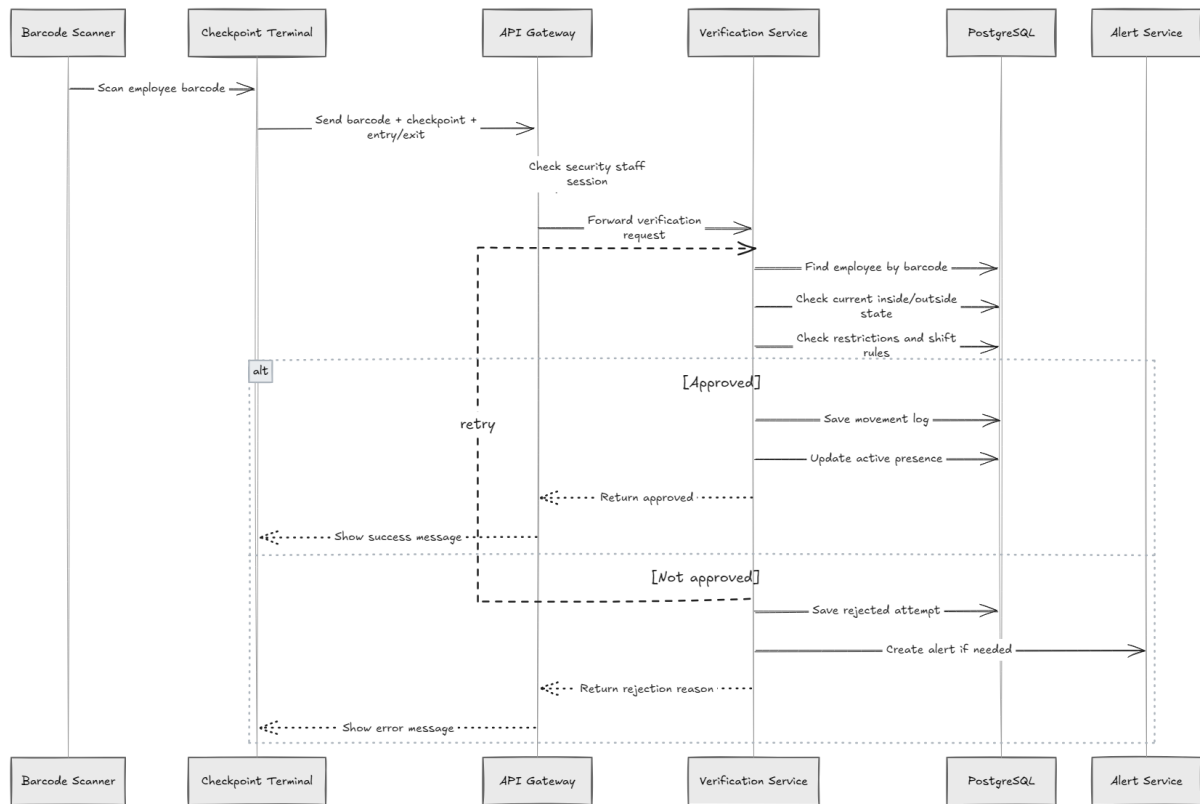


Figure 4: Security Staff Use Case

9.17.2 Admin dashboard use case

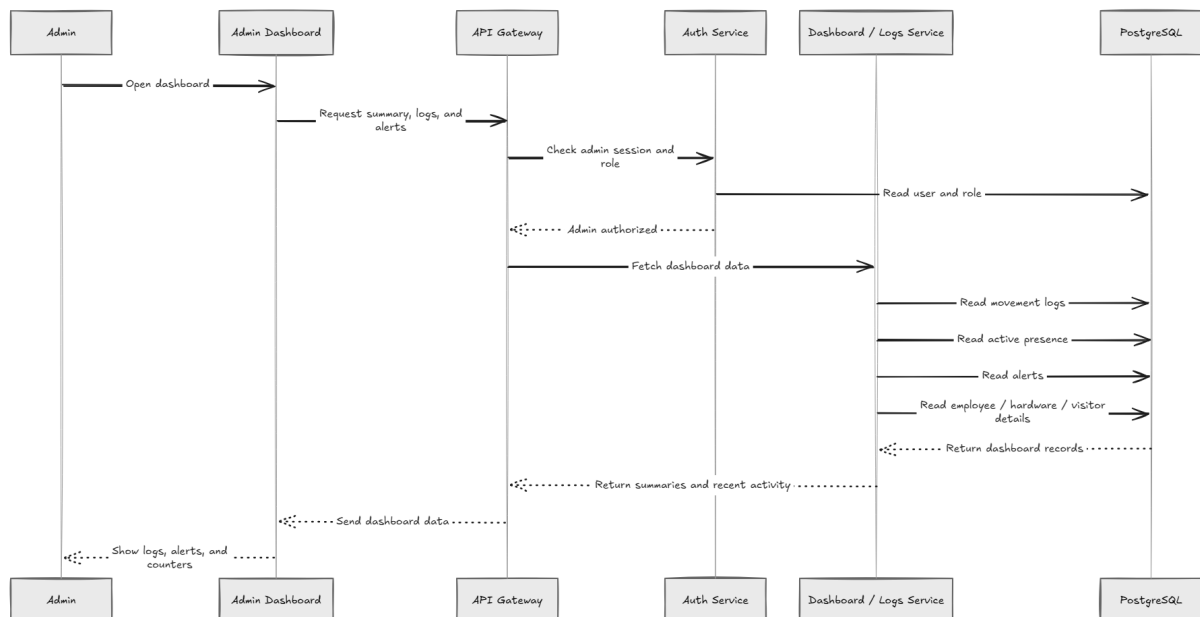


Figure 5: Admin Dashboard Use Case

10 Potential Deep Dives

10.1 1. How are duplicate scans and conflicting active entries prevented?

The movement service writes the log and updates active presence in one database transaction. Unique indexes ensure each employee, visitor, or hardware item has only one current active-presence row. The API also uses idempotency keys so retries do not create duplicate movement transactions.

10.2 2. How do we support offline verification without corrupting audit logs?

The backend gives each registered scanner device a time-limited cache of allowed employees, visitors, hardware items, shift rules, restrictions, and checkpoint rules. Offline decisions are labelled with source `offline`. During sync, the sync service stores the batch, replays events through the same verification rules, records untrusted client capture time separately from server time, detects conflicts, and creates alerts for anything that needs manual review.

10.3 3. How do we keep authentication and local storage secure?

Use OIDC for SSO. Local storage should hold only theme preference, non-sensitive UI state, and bounded offline verification cache metadata. Offline cache data should be signed, encrypted at rest where possible, scoped to a checkpoint and device, and expire quickly.

10.4 4. What happens when scanners, power, or backend services fail?

Unsynced transactions are stored in IndexedDB with durable idempotency keys. If the backend is unavailable, the API Gateway returns a clear degraded-mode response and the UI moves into offline capture mode. When connectivity returns, the sync service records the batch, validates each event, updates active state only when safe, and creates alerts for conflicts.

10.5 5. Why are users and employees separate but linked?

`users` represents login identity and permissions. `employees` represents a real person whose movement can be tracked. A security staff member can be both: they log in through `users`, and their own gate movement is tracked through `employees`. The nullable `employees.user_id` foreign key connects those identities without forcing every employee to have a login.